

Zero to Hero Study Handbook: CUA Workbench

Module 1: Foundations & Architecture

What this project does

CUA Workbench is a local operator console for running computer-using agents inside a visible Linux sandbox. The project combines:

- A React frontend for task input, session control, and live observability (`frontend/src/pages/Workbench.jsx`).
- A FastAPI backend that validates inputs, orchestrates sessions, streams events, and manages Docker lifecycle (`backend/api/server.py`).
- A Dockerized Ubuntu desktop with an internal agent service that executes browser or desktop actions (`docker/entrypoint.sh`, `docker/agent\_service.py`).

Main use cases implemented in this repo:

- Start AI agent sessions from plain-language tasks.
- Choose provider/model/engine per session (`google`, `anthropic`, `openai`).
- Execute one of three engines: `playwright\_mcp`, `omni\_accessibility`, `computer\_use`.
- Inspect execution in real time via timeline/log/screenshot/noVNC streaming.
- Export session artifacts from the frontend (`handleExportSession` in `frontend/src/pages/Workbench.jsx`).

Core paradigms and patterns used here

Agent orchestration loop: The core runtime is a perceive -> think -> act loop implemented by `AgentLoop.run()` and `AgentLoop.\_execute\_step()` in `backend/agent/loop.py`.

Provider strategy pattern: Model inference is provider-routed by `query\_model()` in `backend/agent/model\_router.py`, which delegates to `query\_gemini`, `query\_claude`, or `OpenAICUClient`.

Command dispatch pattern: Action execution is centralized in `execute\_action()` (`backend/agent/executor.py`) and forwarded to engine-specific executors (`playwright\_mcp\_client`, `accessibility\_engine`, or CU executors).

Schema-first validation: Action and engine validity is controlled by `ActionType` (`backend/models.py`) plus `engine\_capabilities.json` through `EngineCapabilities` (`backend/engine\_capabilities.py`).

Event-driven streaming: Frontend state is updated from WebSocket event pushes (`frontend/src/hooks/useWebSocket.js`) sourced by backend `\_broadcast()` and `/ws` in `backend/api/server.py`.

Security-by-default middleware and token gates: Request IDs, no-store key endpoints, origin-checked WebSockets, short-lived WS tokens, and bearer-auth to in-container service are all explicit in code (`backend/api/server.py`, `backend/utils/agent\_auth.py`, `docker/agent\_service.py`).

Architecture description

Key runtime components and interaction:

- Frontend sends REST requests through `frontend/src/api.js` to `/api/\*`.
- Backend API validates session/model/provider/engine, resolves API keys, and creates an `AgentLoop`.
- `AgentLoop` gathers perception (screenshot or AX snapshot), queries model provider, executes action, appends step history.
- Execution layer forwards actions to Playwright MCP, accessibility engine, or native Computer Use engine.
- Docker manager starts/stops sandbox and supplies service bearer token.
- WebSocket channel streams logs/steps/screenshots/safety prompts to the frontend.
- noVNC is reverse-proxied through backend (`/vnc/\*`) for interactive desktop viewing.

ASCII main flow:

```
``text
```

```
User (Workbench UI)
```

```

|
| POST /api/agent/start
v
FastAPI backend (backend/api/server.py)
| validate model/provider/engine + resolve_api_key + start_container
v
AgentLoop (backend/agent/loop.py)
|-- perceive: capture_screenshot() or mcp_get_accessibility_tree()
|-- think: query_model() -> gemini/claude/openai
|-- act: execute_action()
v
Execution adapters
|-- Playwright MCP client (backend/agent/playwright_mcp_client.py)
|-- Accessibility engine (backend/engines/accessibility_engine.py)
|-- ComputerUseEngine (backend/engines/computer_use_engine.py)
v
In-container agent service (docker/agent_service.py)
|-- Playwright browser mode
|-- xdotool desktop mode
v
Sandbox desktop/browser state changes

Parallel event channel:
AgentLoop callbacks -> _broadcast() -> WebSocket /ws -> useWebSocket() ->
timeline/log/screen UI
```

```

## ## Module 2: Repository Map

| File/Directory Path                      | Primary Responsibility                                                               | Key Classes/Functions                                                                                            | Important Configs/Variables                                                                    |
|------------------------------------------|--------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| `backend/main.py`                        | Backend process entrypoint                                                           | `main()` -> `uvicorn.run("backend.api.server:app", ...)`                                                         | `config.host`, `config.port`, `config.debug`                                                   |
| `backend/api/server.py`                  | FastAPI app, REST endpoints, WebSocket, VNC proxy, session registry                  | `lifespan`, `api_start_agent`, `api_stop_agent`, `websocket_endpoint`, `broadcast`, `error_response`             | `MAX_CONCURRENT_SESSIONS`, `MAX_STEPS_HARD_CAP`, `ALLOWED_ORIGINS`, `WS_TOKEN_TTL_SECONDS`     |
| `backend/config.py`                      | Environment-backed runtime config and API key resolution                             | `Config.from_env()`, `resolve_api_key()`, `get_all_key_statuses()`                                               | `GEMINI_MODEL`, `OPENAI_BASE_URL`, `PLAYWRIGHT_MCP_*`, `SCREEN_*`, `MAX_STEPS`, `STEP_TIMEOUT` |
| `backend/models.py`                      | Shared typed contracts for actions/sessions/API payloads                             | `ActionType`, `AgentAction`, `StartTaskRequest`, `TaskStatusResponse`, `StepRecord`, `TaskState`                 | `StartTaskRequest.max_steps <= 200`, `TaskState.COMPLETION_THRESHOLD=3`                        |
| `backend/agent/loop.py`                  | Core session orchestration loop                                                      | `AgentLoop.run()`, `execute_step()`, `run_computer_use_engine()`, `detect_stuck()`, `detect_duplicate_results()` | `MAX_CONSECUTIVE_ERRORS`, `MAX_STUCK_DETECTIONS`, `MAX_DUPLICATE_RESULTS`                      |
| `backend/agent/model_router.py`          | Provider-level model routing                                                         | `query_model()`                                                                                                  | `provider` switch: `google`/`anthropic`/`openai`                                               |
| `backend/agent/gemini_client.py`         | Gemini prompt assembly, call, parse, retry                                           | `build_contents()`, `parse_action()`, `query_gemini()`                                                           | `config.gemini_retry_attempts`, `config.gemini_retry_delay`                                    |
| `backend/agent/anthropic_client.py`      | Claude prompt assembly, call, parse, retry                                           | `build_messages()`, `parse_action()`, `query_claude()`                                                           | prompt cache threshold `PROMPT_CACHE_MIN_CHARS=4000`                                           |
| `backend/agent/openai_client.py`         | OpenAI Responses API computer tool integration                                       | `OpenAICUClient.query()`, `turn_to_legacy_result()`, `extract_turn()`                                            | `store=False`, `tools=[{"type": "computer"}]`, `parallel_tool_calls=False`                     |
| `backend/agent/executor.py`              | Engine-validated action dispatch                                                     | `execute_action()`, `validate_unified_action()`, `send_with_retry()`                                             | `execution_target` (`local`/`docker`), `config.action_delay_ms`                                |
| `backend/agent/playwright_mcp_client.py` | Playwright MCP transports (local STDI0, Docker HTTP), tool discovery, ref resolution | `set_mcp_target()`, `ensure_mcp_initialized()`,                                                                  |                                                                                                |

```

`ensure_docker_mcp_initialized()`, `execute_mcp_action()` | `_DISALLOWED_MCP_TOOLS`=
{"browser_run_code"}`, `PLAYWRIGHT_MCP_HOST/PORT/PATH` |
| `backend/agent/prompts.py` | Engine/provider-specific system prompts |
`get_system_prompt()`, `build_dynamic_mcp_prompt()` | dynamic prompt from discovered MCP
tools |
| `backend/agent/screenshot.py` | Screenshot capture through agent service and fallback |
`capture_screenshot()`, `_fallback_docker_screenshot()`, `check_service_health()` |
`config.agent_service_url`, session-scoped screenshot temp names |
| `backend/engines/computer_use_engine.py` | Native CU protocol runtime for
Gemini/Claude/OpenAI | `ComputerUseEngine.execute_task()`, `GeminiCUClient.run_loop()`,
`ClaudeCUClient.run_loop()`, `_run_openai_loop()` | `Provider`, `Environment`,
`DEFAULT_TURN_LIMIT` |
| `backend/engines/accessibility_engine.py` | Cross-platform accessibility automation
abstraction + handlers | `execute_accessibility_action()`, `ALLY_TOOL_HANDLERS`,
`dump_tree()`, `click_element()` | `CircuitBreaker`, `TTLCache`, strict command allowlist in
`_h_run_command` |
| `backend/engine_capabilities.json` | Single source of truth for engine/action capabilities
| consumed by `EngineCapabilities` | `engines.playwright_mcp`, `engines.omni_accessibility`,
`engines.computer_use` |
| `backend/allowed_models.json` | Canonical provider/model allowlist and CU metadata |
loaded by `_load_allowed_models()` | `supports_computer_use`, `supports_playwright_mcp`,
`cu_tool_version`, `cu_betas` |
| `backend/utils/docker_manager.py` | Docker build/start/stop/status + token extraction |
`start_container()`, `stop_container()`, `get_container_status()` | `config.container_name`,
token file tracking `_tracked_secret_files` |
| `backend/utils/agent_auth.py` | Bearer-token header generation for in-container service |
`set_token_path()`, `get_auth_headers()` | in-memory `_token` from extracted secret file |
| `docker/entrypoint.sh` | Container boot sequence (Xvfb, XFCE, Dbus, AT-SPI, VNC, MCP,
agent service) | shell startup phases + `exec ... agent_service.py` | `DISPLAY=:99`,
`AGENT_SERVICE_TOKEN_FILE`, `PLAYWRIGHT_MCP_PORT` |
| `docker/agent_service.py` | In-container HTTP executor for browser/desktop actions |
`AgentHandler.do_GET/do_POST`, `_dispatch_action()`, `_dispatch_browser()`,
`_dispatch_desktop()` | `_MAX_BODY_SIZE`, `_PUBLIC_PATHS`, `_ALLOWED_COMMANDS` |
| `frontend/src/pages/Workbench.jsx` | Primary operator interface state machine |
`handleStart()`, `handleStop()`, `handleSafetyDecision()`, `handleExportSession()` |
`runMode`, `engine`, `executionTarget`, `sessionId`, `maxSteps` |
| `frontend/src/hooks/useWebSocket.js` | Session-scoped WS lifecycle and event consumption |
`connect()`, `subscribeSession()`, `unsubscribeSession()` | `WS_BASE`, `MAX_LOGS`,
`MAX_STEPS` |
| `frontend/src/api.js` | REST client wrapper and typed endpoint calls | `request()`,
`startAgent()`, `issueWsToken()`, `confirmSafety()` | `API_BASE='/api'`, `X-Request-ID`
generation |
| `frontend/src/components/ScreenView.jsx` | noVNC iframe vs screenshot fallback rendering |
`ScreenView` component | `useVnc`, `vncUrl`, `screenshotFormat` |
| `frontend/vite.config.js` | Dev server and API/WS/VNC proxying | Vite config export |
proxies `/api`, `/ws`, `/vnc` -> `localhost:8000` |
| `setup.sh`, `start.sh`, `setup.bat`, `start.bat` | Operator setup/start scripts |
shell/batch command flows | prerequisite checks, dependency installation, process
launch/stop |
| `.github/workflows/ci.yml` | CI quality/security pipeline | jobs `lint`, `backend`,
`shell-smoke`, `frontend`, `pip-audit`, `npm-audit`, `trivy` | Python 3.13, Node 24.x |

```

## ## Module 3: Core Execution Flows

### ### Flow 1: System bootstrap and UI entry

1. `start.sh` (or `start.bat`) starts backend with `python -m backend.main` and frontend with `npm run dev`.
2. `backend/main.py` loads `config` and starts Uvicorn on `backend.api.server:app`.
3. `frontend/src/main.jsx` mounts routes `/` and `/workbench`.
4. `frontend/src/App.jsx` and `useContainerStatus()` poll `/api/container/status` and `/api/health/detailed`.

Key startup data shape:

```
- `GET /api/container/status` returns `{ "name": str, "running": bool, "image": str,
"agent_service": bool }`.
```

### Flow 2: Start agent session from Workbench

1. User fills task/settings in `Workbench.jsx` and triggers `handleStart()`.
2. Frontend optionally calls `getPreflight(engine, provider)` (`GET /api/preflight`).
3. Frontend calls `startAgent(...)` (`POST /api/agent/start`) with request body mapped from UI state.
4. Backend `api\_start\_agent()` validates engine/provider/model/task, resolves API key via `resolve\_api\_key()`, enforces concurrency/rate limits, and starts container if needed.
5. Backend instantiates `AgentLoop(...)` with scoped callbacks and stores it in `\_active\_loops`.
6. Backend starts async task `\_run\_and\_notify()` that runs `loop.run()` and later broadcasts `agent\_finished`.
7. Frontend stores `session\_id`, marks run as active, then calls `subscribeSession(session\_id)`.
8. `useWebSocket.connect()` fetches token from `POST /api/session/ws-token` and opens `/ws?token=...`, then sends `{"type": "subscribe", "session\_id": "..."}`.

Exact start request shape (`StartTaskRequest`):

```
```json
{
  "task": "Search Google for latest AI news",
  "api_key": "...optional when using saved key...",
  "model": "gemini-3-flash-preview",
  "max_steps": 50,
  "mode": "browser",
  "engine": "playwright_mcp",
  "provider": "google",
  "execution_target": "local",
  "system_prompt": null,
  "allowed_domains": null
}
```
```

Exact start response shape from `api\_start\_agent()`:

```
```json
{
  "session_id": "<uuid>",
  "status": "running",
  "mode": "browser",
  "engine": "playwright_mcp",
  "provider": "google"
}
```
```

### Flow 3: Non-Computer-Use engine step loop (`playwright\_mcp` and `omni\_accessibility`)

Main runtime is `AgentLoop.run()` and `AgentLoop.\_execute\_step()`.

Per-step lifecycle:

1. Perceive.
  - If engine is `playwright\_mcp`: `mcp\_get\_accessibility\_tree()` (text snapshot).
  - Else: `capture\_screenshot(mode, engine)` (base64 PNG).
2. Think.
  - `query\_model(...)` in `model\_router.py` sends context to selected provider client.
  - Provider client parses model JSON into `AgentAction`.
3. Act.
  - `execute\_action(action, mode, engine, execution\_target, session\_id)` validates engine/action, normalizes aliases, and dispatches.
4. Record and stream.
  - Build `StepRecord`, append to `session.steps`, and trigger callbacks for WS fanout.

## 5. Loop controls.

- Stop on `ActionType.DONE`/`ActionType.ERROR`, timeout (`config.step\_timeout`), stuck detection, duplicate-result detection, or max steps.

Key step record shape (`StepRecord`):

```
```json
{
  "step_number": 7,
  "timestamp": "2026-06-30T...Z",
  "screenshot_b64": "...optional...",
  "action": {
    "action": "click",
    "target": "Search button",
    "coordinates": [701, 402],
    "text": null,
    "reasoning": "...",
    "tool_args": null
  },
  "raw_model_response": "{...}",
  "error": null
},
},
```
```

## ### Flow 4: Native `computer\_use` engine

When `engine == "computer\_use"`, `AgentLoop.run()` delegates to `\_run\_computer\_use\_engine()`.

Execution path:

1. Build `ComputerUseEngine(provider, environment, model, tool\_version/beta\_flag for Claude, openai\_base\_url for OpenAI)`.
2. For browser environment, acquire Playwright page via `\_acquire\_playwright\_page()` (CDP first, local browser fallback).
3. Call `ComputerUseEngine.execute\_task(goal, page, on\_safety, on\_turn, on\_log)`.
4. Provider-specific loops:
  - Gemini: `GeminiCUClient.run\_loop()` with `types.Tool(computer\_use=...)`.
  - Claude: `ClaudeCUClient.run\_loop()` with beta tools metadata from `allowed\_models.json`.
  - OpenAI: `\_run\_openai\_loop()` using `OpenAICUClient.query()` and replaying `computer\_call\_output` screenshots.
5. Each CU turn is converted into `CUTurnRecord`, then mapped back to timeline-compatible `StepRecord` via `\_on\_turn` callback in `AgentLoop.\_run\_computer\_use\_engine()`.

Safety confirmation path:

- CU provider emits confirmation requirement.
- `AgentLoop` callback `\_on\_safety()` broadcasts WS `safety\_confirmation` event.
- Frontend calls `POST /api/agent/safety-confirm` with `{ "session\_id": "...", "confirm": true|false }`.
- Loop waits up to 30 seconds; timeout auto-denies.

## ### Flow 5: Action execution dispatch and in-container execution

For non-CU actions, dispatch is split between backend and in-container service.

Backend dispatch (`backend/agent/executor.py`):

- `playwright\_mcp` -> `execute\_mcp\_action(...)` or direct passthrough with `tool\_args`.
- `omni\_accessibility` with `execution\_target="local"` -> `execute\_accessibility\_action(...)`.
- `omni\_accessibility` with `execution\_target="docker"` -> POST to agent service `/action`.

In-container dispatch (`docker/agent\_service.py`):

- `POST /action` parses `{action, coordinates, text, target, mode}`.
- `\_dispatch\_browser(...)` maps to Playwright helper functions (`\_pw\_click`, `\_pw\_type`, `\_pw\_navigate`, etc.).
- `\_dispatch\_desktop(...)` maps to xdotool/wmctrl helpers (`\_xdo\_click`, `\_xdo\_type`, `\_xdo\_open\_url`, etc.).

Agent service action payload shape:

```
```json
{
  "action": "click",
  "coordinates": [500, 320],
  "text": "",
  "target": "",
  "mode": "desktop"
}
```
```

Typical action result shape:

```
```json
{
  "success": true,
  "message": "Clicked at (500, 320)"
}
```
```

### ### Flow 6: Real-time event streaming and VNC

WebSocket (`/ws`) event contract used by frontend `useWebSocket.js`:

- Client messages: `{ "type": "ping" }`, `{ "type": "subscribe", "session\_id": "<uuid>" }`, `{ "type": "unsubscribe", "session\_id": "<uuid>" }`.
- Server events: `log`, `step`, `screenshot`, `screenshot\_stream`, `agent\_finished`, `safety\_confirmation`, `pong`.

Examples:

```
```json
{"event": "log", "log": { "level": "info", "message": "...", "session_id": "<uuid>" }
{"event": "step", "step": { "step_number": 3, "action": { "action": "click" } }, "session_id": "<uuid>" }
{"event": "agent_finished", "session_id": "<uuid>", "status": "completed", "steps": 12}
```
```

VNC path:

- Frontend `ScreenView` calls `issueWsToken(sessionId)`.
- Builds iframe URL `/vnc/vnc.html?...&path=vnc/websockify?token=...`.
- Backend proxies `/vnc/{path:path}` and WS `/vnc/websockify` to internal noVNC server.

## ## Module 4: Setup & Run Guide

### ### Install prerequisites

Minimum expected from repo manifests/scripts:

- Docker with running daemon.
- Python 3.10+ (CI uses 3.13).
- Node.js 24+ (enforced by `frontend/package.json` engines and CI matrix).
- `uv` required by `setup.sh`/`start.sh` on Linux/macOS.

### ### Installation paths

Option A: Scripted setup.

```
```bash
bash setup.sh
```
```

Windows:

```
```bat
setup.bat
```
```

What setup scripts do in this repo:

- Validate prerequisites and Docker daemon.

- Build image via `docker compose build`.
- Create `.env` and install Python deps.
- Install frontend deps in `frontend/`.

Option B: Manual setup.

```
```bash
docker compose build
python -m venv .env
source .env/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
cd frontend && npm install
```
```

### Environment configuration

Use `.env` at repo root (loaded by `backend/config.py` via `load\_dotenv(..., override=False)`).

Most important keys to set first:

- `GOOGLE\_API\_KEY`
- `ANTHROPIC\_API\_KEY`
- `OPENAI\_API\_KEY`
- `OPENAI\_BASE\_URL` (optional custom OpenAI-compatible endpoint)

Runtime control keys actually loaded by `Config.from\_env()`:

- `GEMINI\_MODEL`
- `CONTAINER\_NAME`
- `AGENT\_SERVICE\_HOST`, `AGENT\_SERVICE\_PORT`, `AGENT\_MODE`
- `PLAYWRIGHT\_MCP\_HOST`, `PLAYWRIGHT\_MCP\_PORT`, `PLAYWRIGHT\_MCP\_PATH`
- `PLAYWRIGHT\_MCP\_AUTOSTART`, `PLAYWRIGHT\_MCP\_COMMAND`, `PLAYWRIGHT\_MCP\_ARGS`
- `PLAYWRIGHT\_MCP\_DOCKER\_TRANSPORT`
- `SCREEN\_WIDTH`, `SCREEN\_HEIGHT`, `SCREENSHOT\_FORMAT`
- `MAX\_STEPS`, `ACTION\_DELAY\_MS`, `STEP\_TIMEOUT`, `GEMINI\_RETRY\_ATTEMPTS`
- `HOST`, `PORT`, `DEBUG`
- `VNC\_PASSWORD`

Model and engine policy configuration files:

- `backend/allowed\_models.json`
- `backend/engine\_capabilities.json`

### Running the project

Recommended launcher path:

```
```bash
./start.sh
```
```

Windows:

```
```bat
start.bat
```
```

Stop launcher-managed backend/frontend:

```
```bash
./start.sh --stop
```
```

Container lifecycle in this repo:

- Preferred from UI through header buttons (`Start Sandbox`/`Stop Sandbox`).
- Equivalent API endpoints exist: `/api/container/start`, `/api/container/stop`.

### Database, migrations, seeding

This repository has no database migration or seed system in the current codebase.

- Session state is in-memory (`\_active\_loops`, `\_active\_tasks` in `backend/api/server.py`).
- No ORM, migration tool, or persistent storage layer is present.

### ### Optional extras

WebRTC support is optional and not in default `requirements.txt`.

```
```bash
python -m pip install aiortc av
```
```

Without these extras, `POST /webrtc/offer` returns `501` with an install hint.

### Export this handbook to PDF  
From the repo root:

```
```bash
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf
```
```

Optional A4 formatting:

```
```bash
pandoc ZERO_TO_HERO_STUDY_HANDBOOK.md -o ZERO_TO_HERO_STUDY_HANDBOOK.pdf -V geometry:a4paper
-V margin=1in
```
```

## ## Module 5: Study Plan & Practice Exercises

### ### Ordered study plan for new learners

1. Read `README.md` and `docs/USAGE.md` to understand product intent, operator flow, and supported endpoints.
2. Read `frontend/src/pages/Workbench.jsx` and `frontend/src/api.js` to understand how UI state becomes backend requests.
3. Read `backend/api/server.py` end-to-end. This is the control plane for sessions, tokens, WS events, and container lifecycle.
4. Read `backend/models.py` and `backend/config.py` to lock in request/response/data contracts and env behavior.
5. Read `backend/agent/loop.py` and `backend/agent/executor.py` to understand step execution semantics.
6. Read provider clients: `backend/agent/gemini\_client.py`, `backend/agent/anthropic\_client.py`, `backend/agent/openai\_client.py`.
7. Read `backend/engines/computer\_use\_engine.py` for native CU path.
8. Read `backend/agent/playwright\_mcp\_client.py` and `backend/engines/accessibility\_engine.py` for engine-specific details.
9. Read `docker/entrypoint.sh` and `docker/agent\_service.py` to understand the runtime substrate inside the container.
10. Use selected tests (`tests/test\_api\_errors.py`, `tests/test\_ws\_auth.py`, `tests/test\_execution\_target.py`, `tests/test\_openai\_cu\_integration.py`) to verify your mental model.

### ### Practice exercises

1. Exercise: Trace the full lifecycle of `handleStart()` from frontend to first timeline item.

Solution outline: Follow `Workbench.handleStart()` -> `startAgent()` -> `api\_start\_agent()` -> `AgentLoop.run()` -> `\_execute\_step()` -> `\_scoped\_step()` -> WS `step` event -> `useWebSocket` state update.

2. Exercise: Explain how API key precedence works when both `.env` and UI key are present.

Solution outline: Read `resolve\_api\_key()` in `backend/config.py`; order is UI (`ui\_key`) first, then `.env`/env via `\_detect\_key\_source()`, else none.

3. Exercise: Identify where model-engine compatibility is enforced before session start.

Solution outline: In `api\_start\_agent()` (`backend/api/server.py`), model is checked against `\_VALID\_MODELS\_BY\_PROVIDER`, then `\_model\_supports\_engine(model\_entry, req.engine)`.

4. Exercise: Show exactly how WebSocket token/session binding prevents cross-session subscription.

Solution outline: `POST /api/session/ws-token` binds token to session; `/ws` keeps pending token; `subscribe` checks token record `session_id == sid`, consumes token, otherwise closes with policy violation.

5. Exercise: Compare the action path for `click` in `playwright_mcp` vs `desktop` mode.

Solution outline: `execute_action()` routes to MCP (`execute_mcp_action`) for `playwright_mcp`; in desktop path, agent service `_dispatch_desktop` maps `click` to `_xdo_click`.

6. Exercise: Explain how stuck-loop mitigation is implemented.

Solution outline: In `AgentLoop`, see `_detect_stuck()`, `_detect_duplicate_results()`, `_build_recovery_hint()`, `MAX_STUCK_DETECTIONS`, and read-only carveout via `is_read_only_action()`.

7. Exercise: Add a new provider model mentally and list all files that must remain consistent.

Solution outline: Update `backend/allowed_models.json`; ensure frontend `/api/models` dropdown sees it; ensure provider client supports it; for Claude CU include `cu_tool_version` and `cu_betas` because `_build_allowed_model_state()` enforces them.

8. Exercise: Explain how `computer_use` safety confirmations reach the UI and return to loop execution.

Solution outline: CU loop calls `on_safety` in `AgentLoop.run_computer_use_engine()`; backend broadcasts `safety_confirmation`; UI submits `/api/agent/safety-confirm`; loop awaits `_safety_events[sid]` and uses `_safety_decisions[sid]`.

9. Exercise: Decode session export JSON fields in frontend and map each to source state.

Solution outline: In `handleExportSession()`, exported object includes `config`, `steps`, `logs`, `final_screenshot`; values come from component state and WebSocket hook state.

10. Exercise: Describe why `browser_run_code` is blocked even if MCP server exposes it.

Solution outline: `backend/agent/playwright_mcp_client.py` denylists it via `_DISALLOWED_MCP_TOOLS` and rejects calls in `_mcp_call()` for RCE risk.

### ### Learner verification checklist

- Can you explain how `POST /api/agent/start` validates provider/model/engine and resolves credentials?
- Can you describe the exact difference between `playwright_mcp`, `omni_accessibility`, and `computer_use` in this implementation?
- Can you trace one `StepRecord` from creation to WebSocket emission and frontend rendering?
- Can you explain the token/auth model for `/ws`, `/vnc/websockify`, and in-container `/action`?
- Can you map where action aliases are normalized and where unsupported actions are rejected?
- Can you explain how `execution_target` changes dispatch behavior for accessibility and Playwright paths?
- Can you identify which configs are loaded from environment and which are hardcoded defaults?
- Can you describe how safety confirmation timeout is handled for CU engine actions?
- Can you explain where session state lives and what is lost on backend restart?
- Can you list the first five files you would read before changing runtime behavior, and why?